

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name : Ashwin S

Reg no : 24011101007

Class : AI DS - A (batch 1)

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

3. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Preprocessing performed:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to $[0,1]$.
5. Convert labels into one-hot vectors.

4. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

5. Source Code

The complete implementation, including data preprocessing, model construction, training, evaluation, and hyperparameter optimization, is available in the following repository :

<https://github.com/Ashwin271206/dl-lab/tree/main/Lab%202>

The repository contains a single Google Colab notebook covering all tasks experimented in this report, along with the generated figures.

6. Hyperparameter Optimization

Instead of manually selecting hyperparameters, hyperparameter optimization is performed using **RandomizedSearchCV** together with the SciKeras wrapper.

The following hyperparameters were optimized :

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

7. Results

Best Hyperparameters

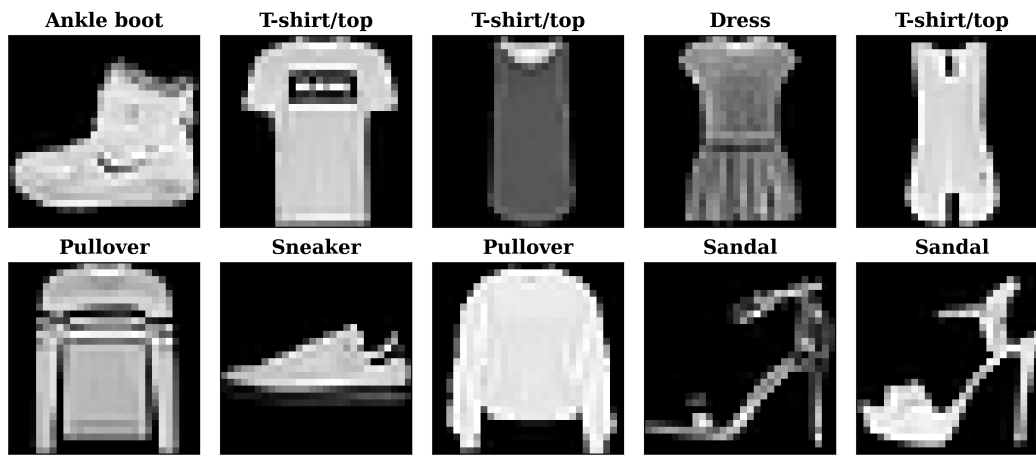
Hidden Layers	2
Hidden Neurons	64
Learning Rate	0.001
Batch Size	128
Optimizer	Adam
Activation Function	Tanh
Epochs	30
Dropout	0.5
Cross-validation Accuracy	0.8257
Testing Accuracy	0.8705

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8840	0.8705
Precision	0.8846	0.8719
Recall	0.8840	0.8705
F1-score	0.8842	0.8709
Training Time (s)	134.51	50.51

8. Plots & Inferences

Sample Images



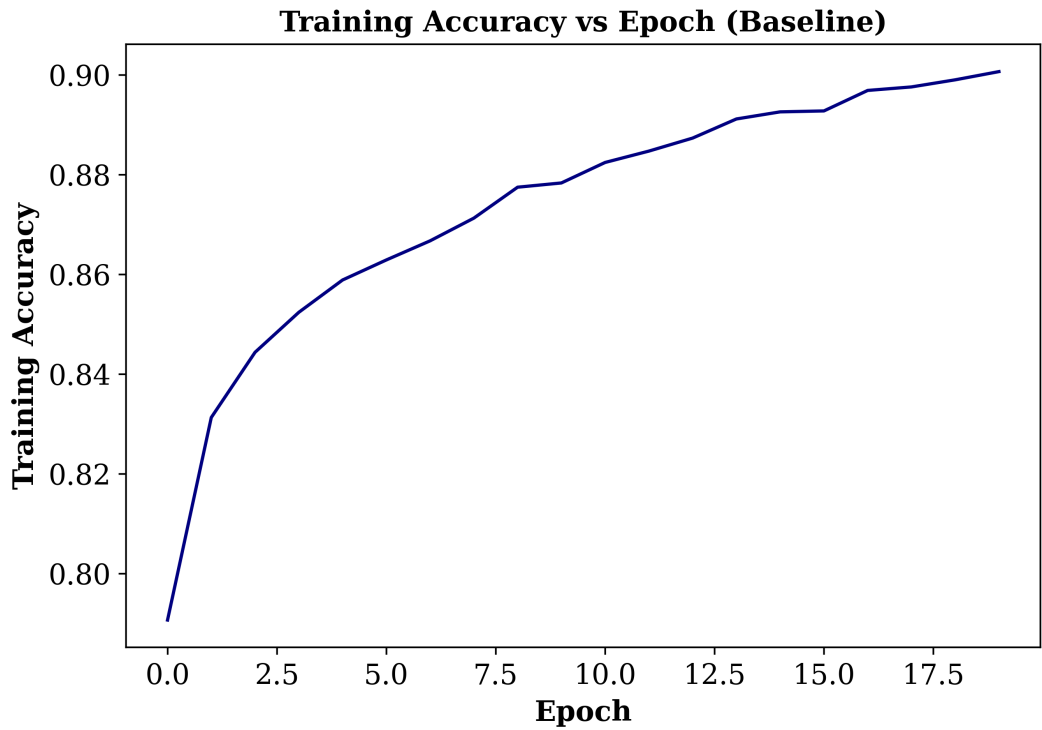
The sample image grid was generated successfully, indicating that the dataset is fully loaded and labelled correctly. These 28 x 28 images look like the mentioned categories such as tops, dresses, sandals, etc, showing that the dataset is fully read in the expected format.

Class Distribution



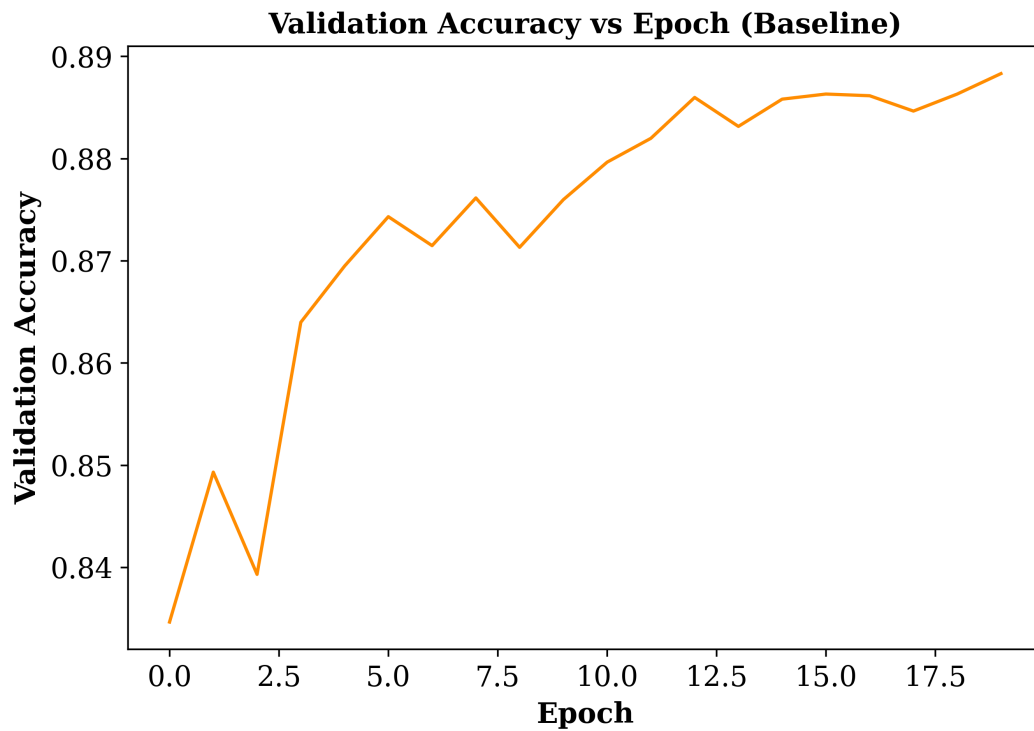
All ten classes have an equal number of training samples as that Fashion-MNIST is a balanced dataset. This equal distribution of classes means that evaluation metrics would be fair for all classes, and the metric of every class matters equally.

Training Accuracy vs Epoch



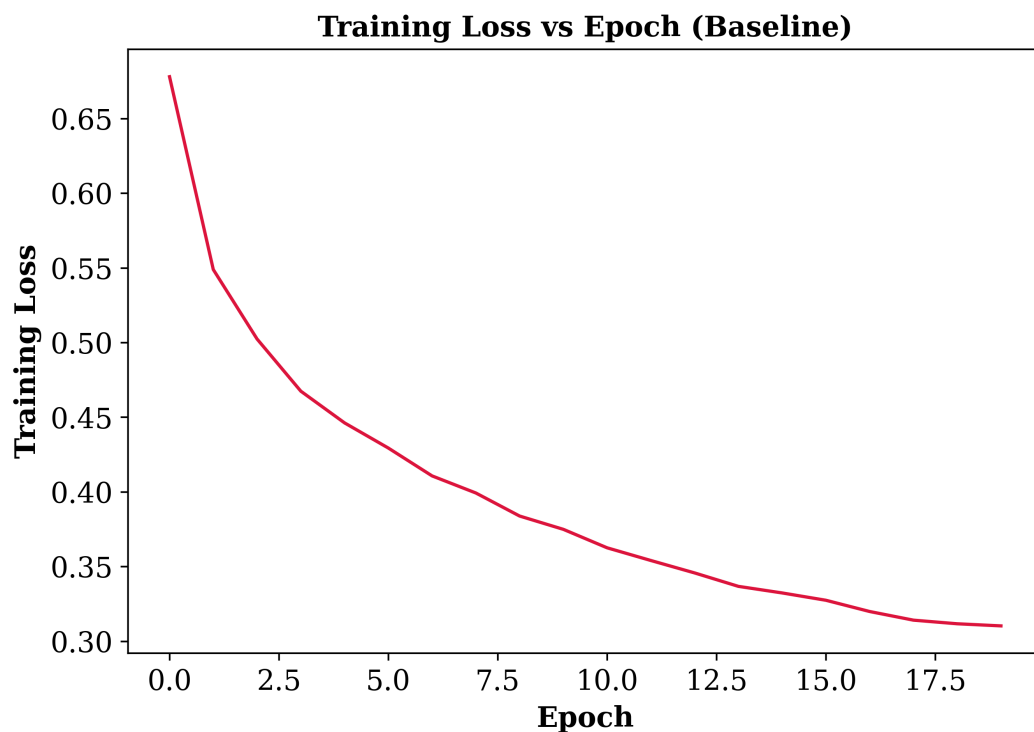
This plot indicates that training accuracy increases steadily across 20 epochs. No irregular spikes were seen, meaning that the model learns consistently on the training data. The curve does not stabilize completely at 20 epochs, suggesting that the model has some more training capacity beyond 20 epochs..

Validation Accuracy vs Epoch



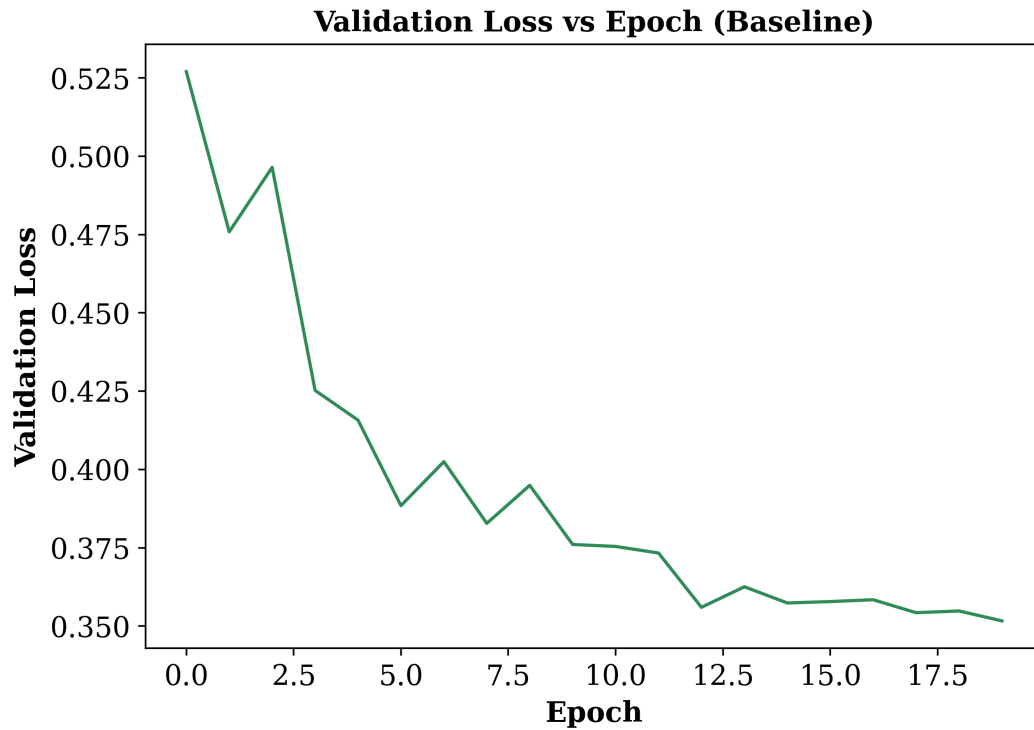
Validation accuracy rises quickly in the first few epochs and then continues to improve more gradually, stabilizes around epoch 12. There was no overfitting detected as accuracy did not drop with later epochs, and show that batch normalization, dropout and L2 decay were effective in stopping overfitting. A few fluctuations were found in the plot.

Training Loss vs Epoch



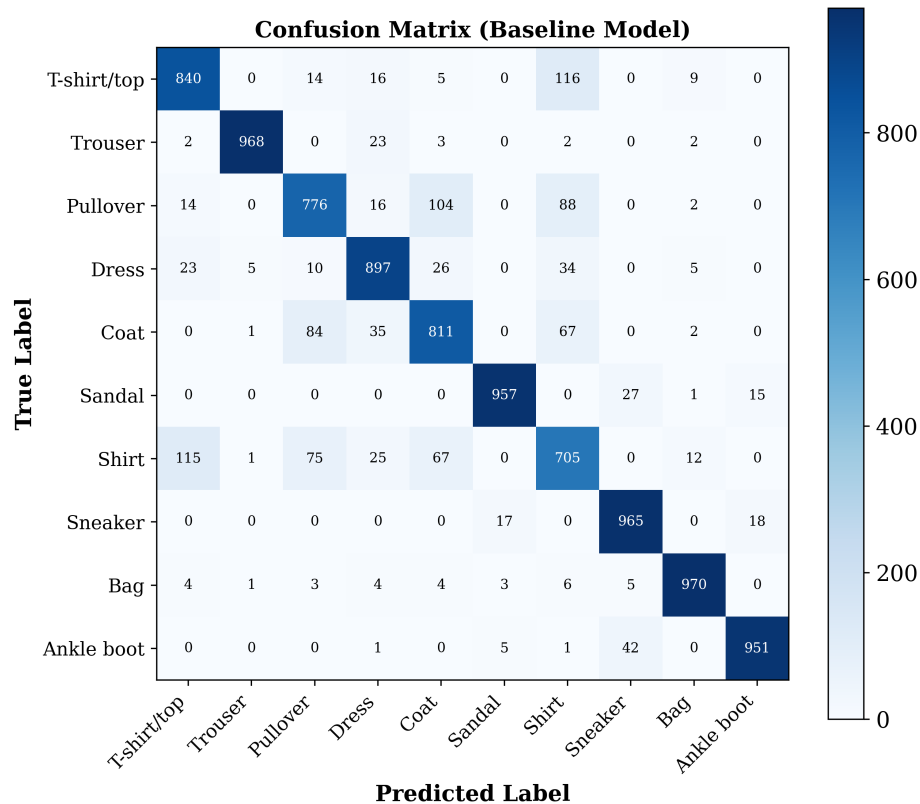
Training loss decreases rapidly in the beginning, and then decreases smoothly and consistently across later epochs. This shows the consistent improvement of the model and is approaching convergence. This aligns with the inferences of previous plots, that the model is improving steadily without any oscillations.

Validation Loss vs Epoch



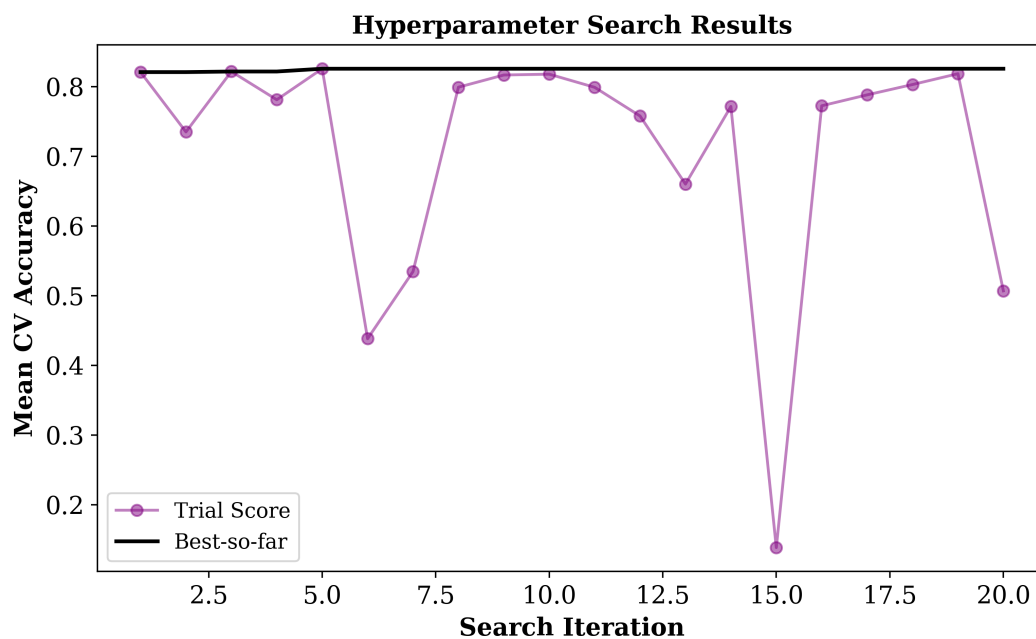
Validation loss decreases almost monotonically from 0.53 to about 0.35, and stabilizes after around epoch 12. This matches with the inference of validation accuracy curve. This indicates that there is no overfitting as there is no spike in later epochs.

Confusion Matrix



The confusion matrix shows strong performance across most classes, and the diagonal is clearly dominant. Most classes are identified correctly. The misclassified samples seem to be similar garments like shirts, coats and pullovers.

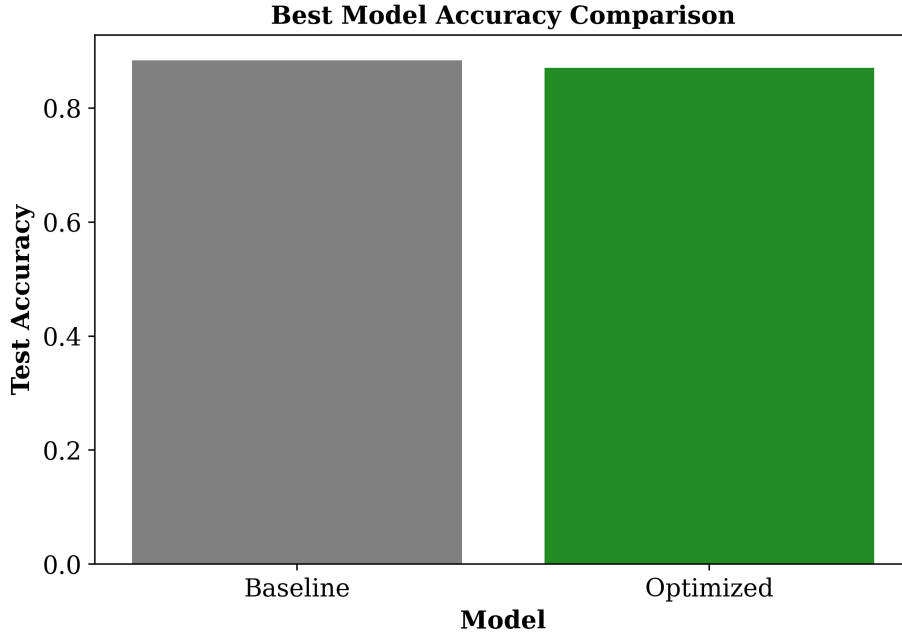
Hyperparameter Search Results



Individual trial scores vary considerably, with a few sharp drops corresponding to poorly performing hyperparameter combinations (for example, SGD with a high learning rate, or sigmoid activation across

multiple layers). The best-so-far line overlaid on the plot shows that a strong configuration was already found within the first ten iterations, after which further search iterations mostly served to confirm rather than substantially improve on it.

Best Model Accuracy Comparison



The bar chart shows the baseline model slightly better optimized model on test accuracy. Hyperparameter optimization here traded a small amount of accuracy for a much faster training time, rather than producing a better model on accuracy alone.

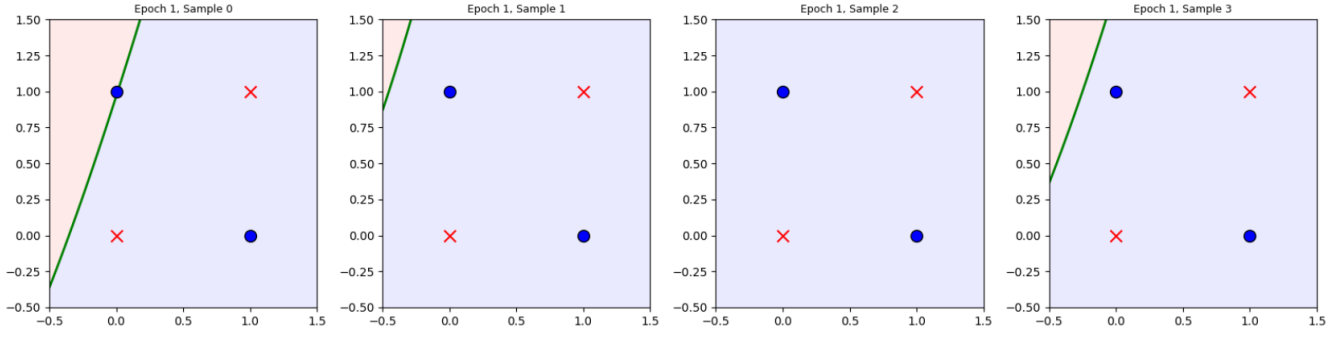
9. Additional Experiment: XOR Gate using a Multi-Layer Perceptron

The previous experiment pointed out that single layer perceptrons cannot solve non-linearly separable problems like the XOR problem. This additional experiment uses a manually implemented multilayer perceptron with forward and backward propagation and weight updates with one hidden layer to learn the XOR function. The multilayer perceptron was trained for 8000 weight updates, and the weights were tracked at each step to observe how the decision boundary gradually reshapes itself from a straight line into a curve that correctly separates the classes. It converges at around 2000 epochs.

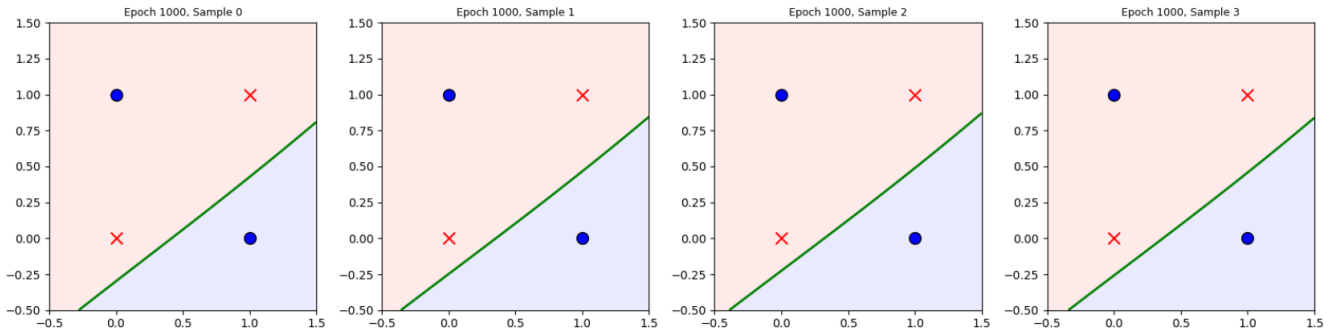
Initial Weights and Biases

Initial W_1	$\begin{bmatrix} -0.251 & 0.901 \\ 0.464 & 0.197 \end{bmatrix}$
Initial b_1	$\begin{bmatrix} -0.688 & -0.688 \end{bmatrix}$
Initial W_2	$\begin{bmatrix} -0.884 \\ 0.732 \end{bmatrix}$
Initial b_2	$\begin{bmatrix} 0.202 \end{bmatrix}$

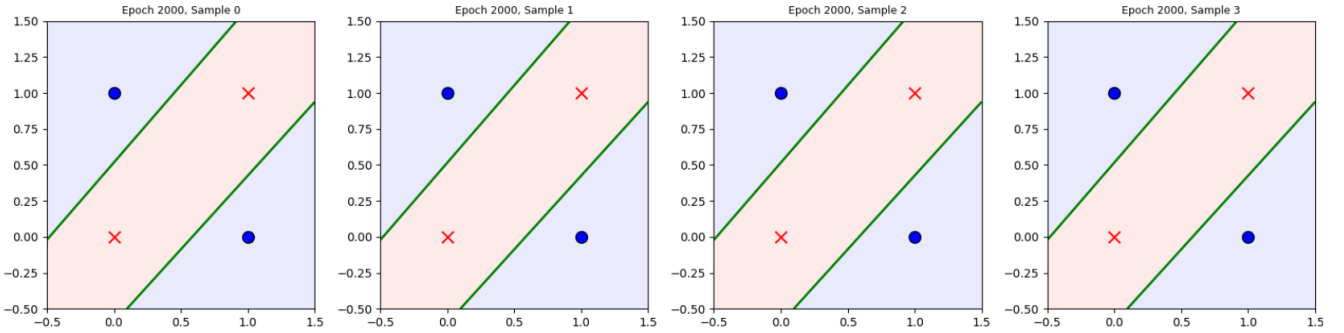
Decision Boundary Evolution



At epoch 1, the decision boundary is still close to its randomly initialized state. It forms a roughly straight line that does not separate the XOR classes correctly.



By epoch 1000, the boundary has started to bend, showing the hidden layer beginning to carve out a non-linear region around the positive class.



By epoch 2000, the decision boundary has curved enough to correctly enclose the two positive XOR points while excluding the two negative ones. This shows that the multilayer perceptron has learned the separation completely.

Final Predictions vs Targets

Input Pattern	(0,0)	(0,1)	(1,0)	(1,1)
Final Prediction	0.064	0.925	0.944	0.056
Target	0	1	1	0

10. Discussion

1. Which optimization method was used?

RandomizedSearchCV was used instead of GridSearchCV. The size of the search grid of hyperparameters (hidden layers, hidden neurons, learning rate, batch size, epochs, optimizer, activation, and dropout rate), a full grid would have required training thousands of model combinations under 5-fold cross-validation, which was not practical using the available computing resources. Hence, RandomizedSearchCV was used. RandomizedSearchCV samples a fixed number of combinations (20) from the space, which gave a reasonable trade-off between search quality and runtime.

2. Which hyperparameters were selected?

The search selected 2 hidden layers of 64 neurons each, a learning rate of 0.001, batch size 128, Adam optimizer, Tanh activation, 30 training epochs, and a dropout rate of 0.5. This is a smaller, more heavily regularized network than the baseline (128 $\rightarrow$ 64), with a fair dropout rate. This shows that the search favoured a configuration that generalizes rather than a configuration that maximizes raw capacity.

3. Did optimization improve performance?

Optimization did not improve in terms of accuracy, the baseline reached 88.40% test accuracy while the optimized model reached 87.05%, so the baseline was slightly better on this metric. However, the optimized model trained in roughly 50 seconds compared to about 134 seconds for the baseline, which is a meaningful efficiency gain for a drop of only about 1.35% accuracy. This is a significant improvement in terms of time.

4. Which hyperparameter had the greatest impact?

Based on how much the search results fluctuated between iterations, the optimizer and learning rate combination appeared to matter the most. Trials using SGD with a high learning rate (0.1) collapsed to very low cross-validation accuracy, while Adam-based trials were consistently strong regardless of other settings. Activation function also played an important role. Sigmoid activation stacked across multiple hidden layers tended to perform poorly, most likely due to vanishing gradients, whereas ReLU and Tanh were both stable. Hidden layer count and neuron width had a comparatively smaller effect on accuracy within the ranges tested.

5. Compare Grid Search and Randomized Search.

Grid Search evaluates every possible combination in the defined search space, which guarantees that the best combination within that space will be found, but the computing cost and time increases rapidly as hyperparameters are added. In this experiment, it is an 8-dimensional space, and grid search would be infeasible in this space. Randomized Search samples a fixed number of random combinations, which does not guarantee finding the single best combination, but in practice tends to find a good combination much faster, especially when only a few of the hyperparameters actually matter much. For this experiment, Randomized Search was the more sensible choice given the size of the search space and the time available.

6. Recommend the best MLP configuration.

Taking both accuracy and training cost into account, the baseline configuration is recommended if test accuracy is the main priority, since it achieved the highest accuracy. If faster training and a smaller, more regularized model are more important, then the optimized configuration (2 layers of 64 neurons, Tanh activation, dropout 0.5, batch size 128) is a reasonable alternative, trading a small amount of accuracy for less than half the training time.

11. Conclusion

This experiment implemented and trained a multilayer perceptron for multi-class image classification on the Fashion-MNIST dataset. It covered the complete implementation from raw image preprocessing through to automated hyperparameter optimization. The baseline model, following the given architecture achieved a test accuracy of 88.40%. An alternative configuration was found through hyperparameter search using RandomizedSearchCV with 5-fold cross-validation. This new model is a smaller two-layer network with Tanh activation and heavier dropout, and produced a slightly lower test accuracy of 87.05% but trained in less than half the time of the baseline model. This is a very huge improvement in terms of time, in exchange for a minor tradeoff on accuracy. The additional experiment used the multilayer perceptron to solve the XOR problem, proving that it can draw non-linear decision boundaries, and solve problems which single layer perceptrons can't. Overall, the experiment demonstrated the complete workflow of building, training, evaluating, and tuning a multilayer perceptron, analyzed its advantages over single layer perceptron, and the importance of hyperparameter tuning.

12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.